

Gaussian Likelihood

Continuous response where the mean trend is a linear combination of the inputs and unknown parameters

$$p(\mathbf{y} | \mathbf{X}, \boldsymbol{\beta}, \sigma^2) = \prod_{n=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{(y_n - \mu_n)^2}{2\sigma^2} \right\} \quad \text{where } \mu_n = \sum_{d=0}^D x_{n,d} \beta_d$$

$$p(\mathbf{y} | \mathbf{X}, \boldsymbol{\beta}, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left\{ -\frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) \right\}$$

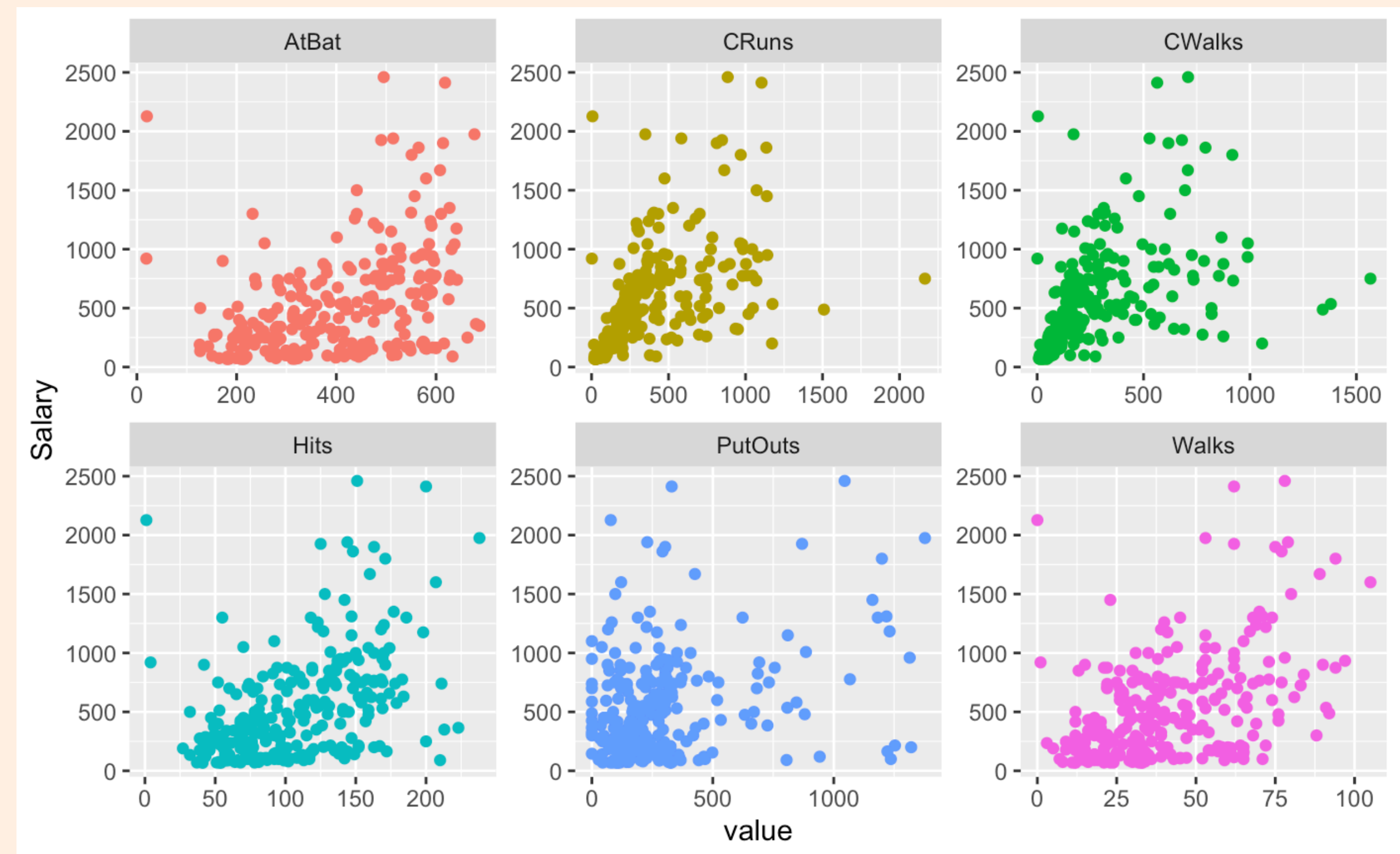
$$\hat{\boldsymbol{\beta}}_{MLE} = \mathbf{X}^T \mathbf{y} (\mathbf{X}^T \mathbf{X})^{-1}$$

Assume that σ , the likelihood/
measurement noise, is known

$$\mathbf{X} = \begin{bmatrix} 1 & x_{1,1} & x_{1,2} & \cdots & x_{1,D} \\ 1 & x_{2,1} & x_{2,2} & \cdots & x_{2,D} \\ 1 & \vdots & \vdots & \cdots & \vdots \\ 1 & x_{N,1} & x_{N,2} & \cdots & x_{N,D} \end{bmatrix} \quad \boldsymbol{\beta} = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_D \end{bmatrix}$$

With linear models, we are trying to learn parameters which define how the response varies with respect to each of the possible inputs.

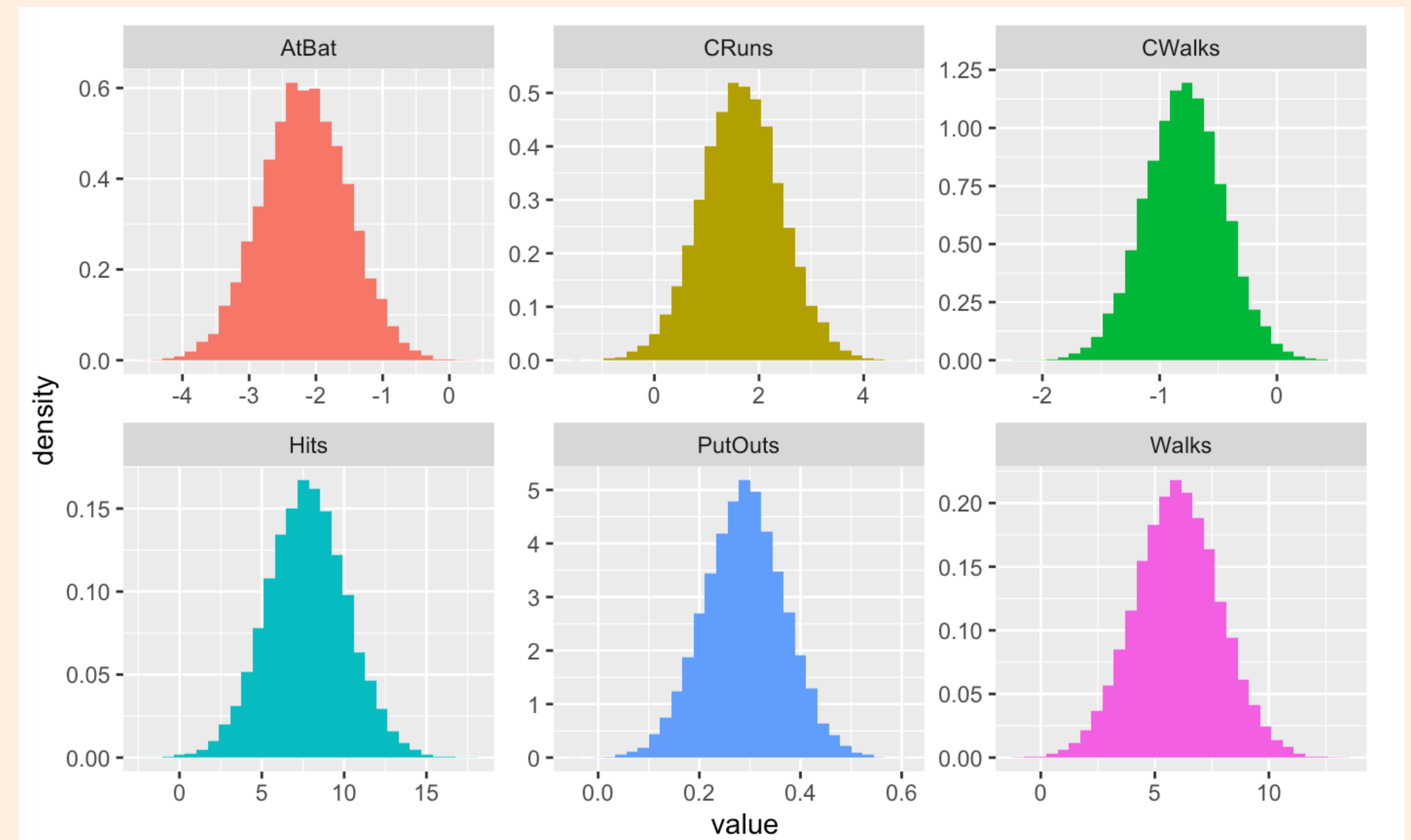
For example, how does a baseball player's salary depend on his performance stats?



- Here I selected a subset of all the features to show their relationship with the response, Salary.
- Some features will be related to the response and others will not. The estimated parameters of the linear model describe how Salary is affected by the number of AtBats, Hits, Walks, etc.
- In maximizing the posterior, we are essentially finding the best-fit-line for every parameter simultaneously

If we assume an infinitely diffuse prior (all values of the parameters are equally likely *a-priori*), then the posterior distribution is proportional to the likelihood

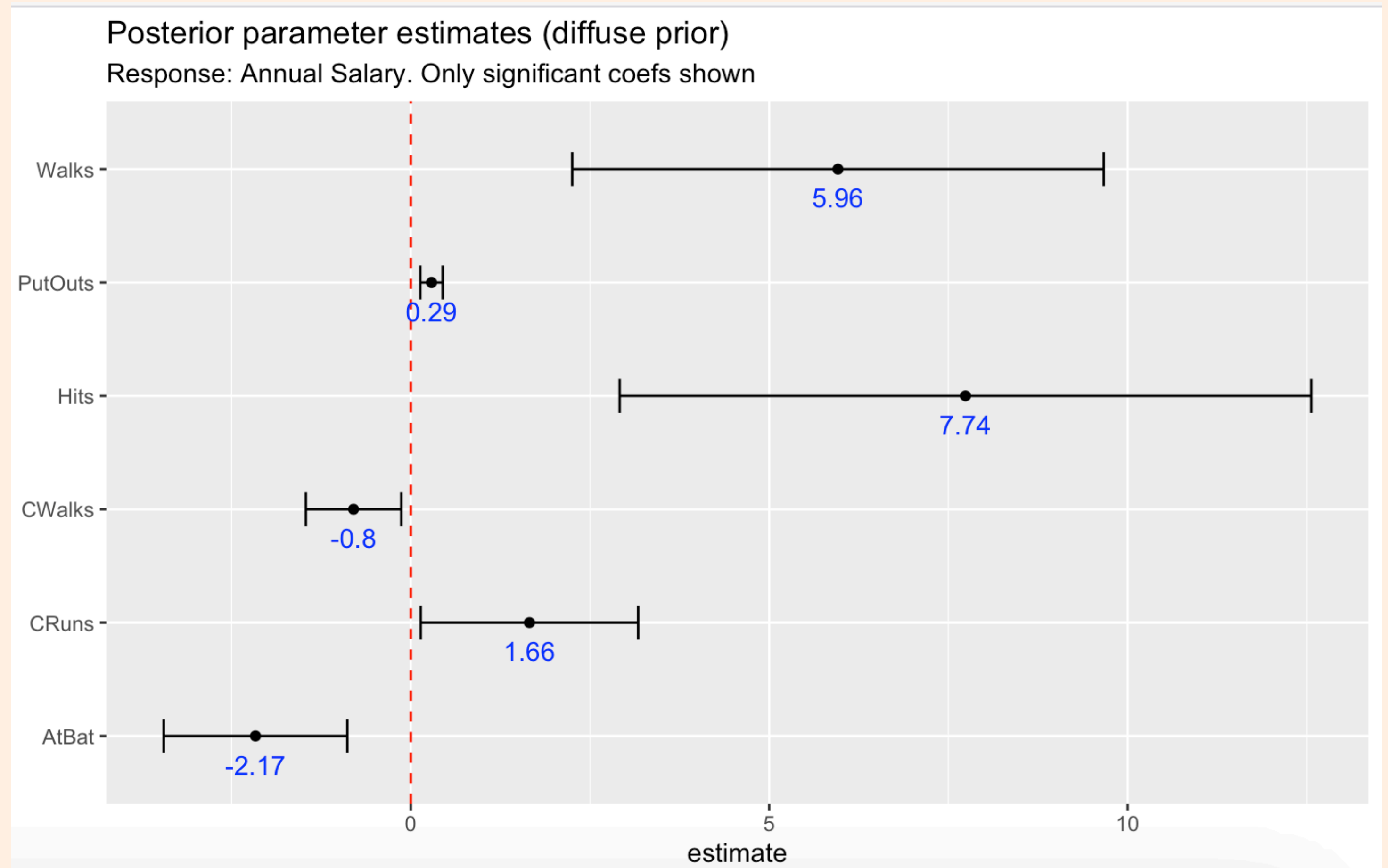
- $p(\boldsymbol{\beta} | \mathbf{y}, \mathbf{X}, \sigma^2) \propto p(\mathbf{y} | \boldsymbol{\beta}, \mathbf{X}, \sigma^2)p(\boldsymbol{\beta})$
- Diffuse prior: $p(\boldsymbol{\beta}) = 1$
- $\mathbb{E}[\boldsymbol{\beta} | \dots] = \mathbf{m}_N = \mathbf{X}^T \mathbf{y} (\mathbf{X}^T \mathbf{X})^{-1}$
- $\text{cov}(\boldsymbol{\beta}, \boldsymbol{\beta}) = \mathbf{S}_N = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}$



What is the meaning of the parameters in the linear model?

Example: predicting baseball players' salary as a function of game performance

- **“Walks” parameter:** each recorded walk is associated with a mean salary *increase* of 5.96 (thousands of \$)
- **“AtBat”:** each at-bat is associated with a salary *decrease* of 2.17



Let's consider an informative prior instead.

What are the reasons to use a prior with linear models?

- Bound or constrain the model's search space - reduce training time by “ruling out” extreme values that are far away from the prior mean.
- Resist overfitting to the data, especially when we have a lot of free parameters

Use a conjugate prior to the Gaussian likelihood

Below is the full definition, which has individual prior means for each parameter and covariances between them

- $\boldsymbol{\beta} \mid \mathbf{m}_0, \mathbf{S}_0 \sim \mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_0, \mathbf{S}_0)$
- $$p(\boldsymbol{\beta} \mid \mathbf{m}_0, \mathbf{S}_0) = \frac{1}{(2\pi)^{(D+1)/2}} \frac{1}{|\mathbf{S}_0|^{1/2}} \exp \left\{ -\frac{1}{2}(\boldsymbol{\beta} - \mathbf{m}_0)^T \mathbf{S}_0^{-1} (\boldsymbol{\beta} - \mathbf{m}_0) \right\}$$
- $\mathbf{m}_0$ is a vector of length $D + 1$, representing the **prior means** for each unknown parameter (including the intercept, which is why I explicitly wrote “+1”)
- $\mathbf{S}_0$ is a $(D + 1) \times (D + 1)$ covariance matrix, representing the **prior variance** (along the diagonal), and covariances (off-diagonals) between parameters

In practice, we'll typically use simpler expressions for the prior means and covariances

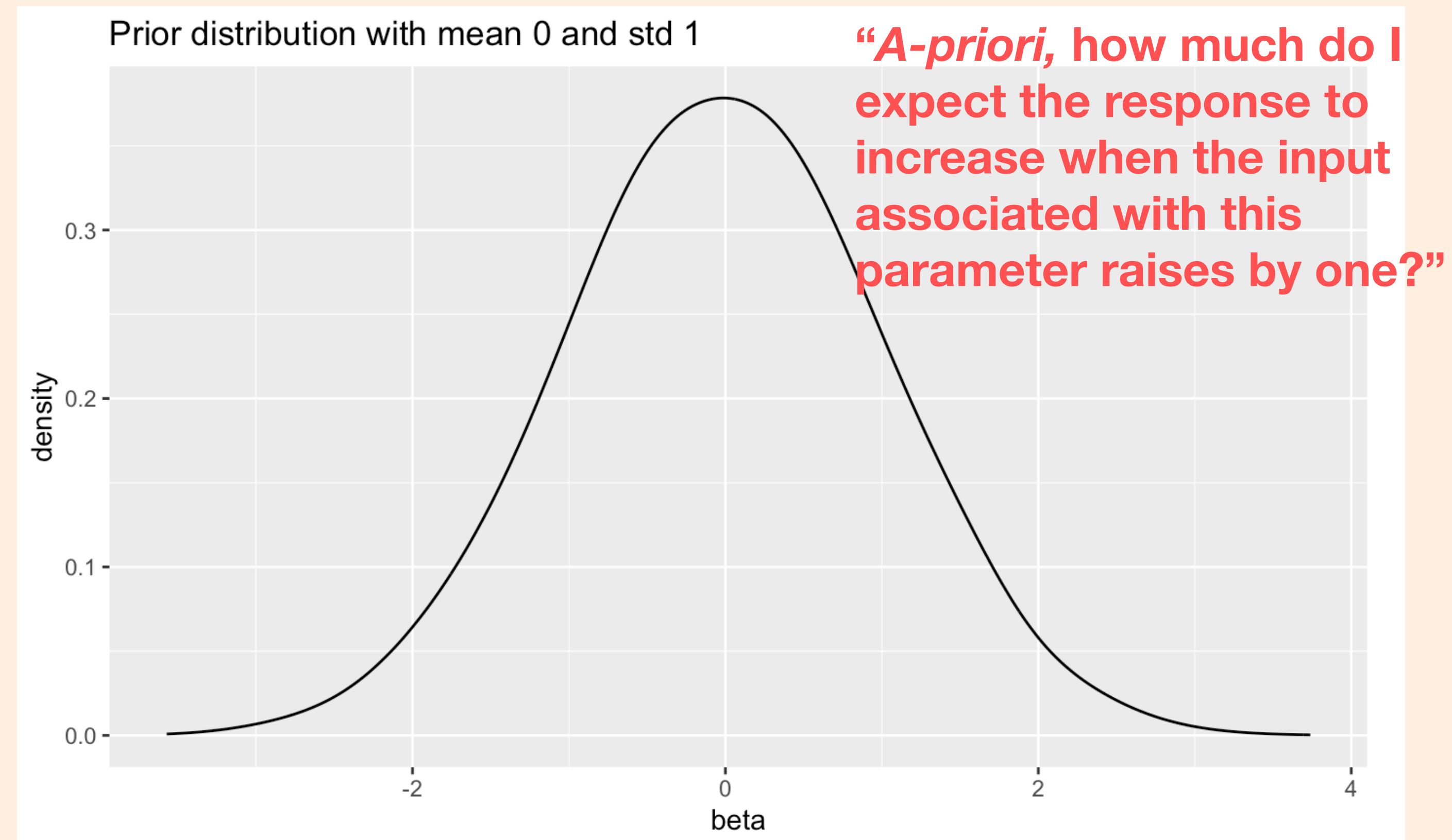
One of the simplest ways: standardize numeric inputs and use a single, shared prior mean and variance.

$$\mathbf{m}_0 = [m_0, m_0, \dots, m_0]$$

$$\mathbf{S}_0 = \mathbb{I}\tau_0 = \begin{bmatrix} \tau_0 & 0 & \dots & 0 \\ 0 & \tau_0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & \tau_0 \end{bmatrix}$$

$$p(\boldsymbol{\beta} \mid m_0, \tau_0) = \prod_{d=0}^D \mathcal{N}(\beta_d \mid m_0, \tau_0)$$

Assuming *a-priori* that parameters are independent and have no correlation structure



Now we have all the information we need to conduct Bayesian Inference

Assuming σ is known, with prior mean and variance m_0 and τ_0^2 respectively

$$\text{Likelihood} \propto \prod_{n=1}^N \exp \left\{ -\frac{(y_n - \mu_n)^2}{2\sigma^2} \right\}$$

$$\mu_n = \sum_{d=0}^D x_{n,d} \beta_d$$

$$\text{Prior} \propto \prod_{d=1}^D \exp \left\{ -\frac{(\beta_d - m_0)^2}{2\tau_0^2} \right\}$$

$$\text{Posterior} \propto \prod_{n=1}^N \exp \left\{ -\frac{(y_n - \mu_n)^2}{2\sigma^2} \right\} \times \prod_{d=0}^D \exp \left\{ -\frac{(\beta_d - m_0)^2}{2\tau_0^2} \right\}$$

After some tedious algebra, you can derive expressions for the posterior mean and covariance

Assuming the posterior is Gaussian...

$$p(\boldsymbol{\beta} \mid \mathbf{y}, \mathbf{X}, \sigma, \mathbf{m}_0, \mathbf{S}_0) = \mathcal{N}(\boldsymbol{\beta} \mid \mathbf{m}_N, \mathbf{S}_N)$$

where

$$\mathbf{m}_N = \mathbf{S}_N(\mathbf{S}_0^{-1}\mathbf{m}_0 + \frac{1}{\sigma^2}\mathbf{X}^T\mathbf{y})$$

$$\mathbf{S}_N^{-1} = \mathbf{S}_0^{-1} + \frac{1}{\sigma^2}\mathbf{X}^T\mathbf{X}$$

If we had a very diffuse prior,
can you see how we could
recover the MLE estimates?

- Fortunately, the mode of a Gaussian distribution is equal to its mode, therefore:

$$\hat{\boldsymbol{\beta}}_{MAP} = \mathbf{S}_N(\mathbf{S}_0^{-1}\mathbf{m}_0 + \frac{1}{\sigma^2}\mathbf{X}^T\mathbf{y})$$

The posterior precision matrix is simply the sum of the data precision and prior precision

$$\mathbf{S}_N^{-1} = \mathbf{S}_0^{-1} + \frac{1}{\sigma^2} \mathbf{X}^T \mathbf{X}$$

and the posterior mean is a precision-weighted average of the data and prior means

$$\mathbf{m}_N = \mathbf{S}_N(\mathbf{S}_0^{-1}\mathbf{m}_0 + \frac{1}{\sigma^2}\mathbf{X}^T\mathbf{y})$$

Things get even simpler if we assume a shared prior mean of 0

Conceptually analogous to a null hypothesis: *a-priori*, the input associated with this parameter has no influence on the response

- Let $\mathbf{m}_0 = [0,0,0\dots]$

$$\bullet \log - \text{Posterior} \propto - \frac{1}{2\sigma^2} \sum_{n=1}^N \{ (y_n - \mu_n)^2 \} - \frac{1}{2\tau_0^2} \sum_{d=0}^D \beta_d^2$$

SSE **Parameter magnitude**

Maximizing the posterior is equivalent to minimizing the SSE, with an additional penalty for the size of the parameter estimates.

In non-Bayesian terms, this is known as “Ridge Regression”

Or even more generally, it's L_2 regularization, which penalizes a model's objective function according to the square of the parameter estimates

- $\hat{\boldsymbol{\beta}} = \arg \min_{\boldsymbol{\beta}} \left\{ \text{SSE}(\mathbf{y}; \mathbf{X}, \boldsymbol{\beta}) + \gamma \sum_{d=0}^D \beta_d^2 \right\}$
- where γ is a constant penalty factor, analogous to the prior precision

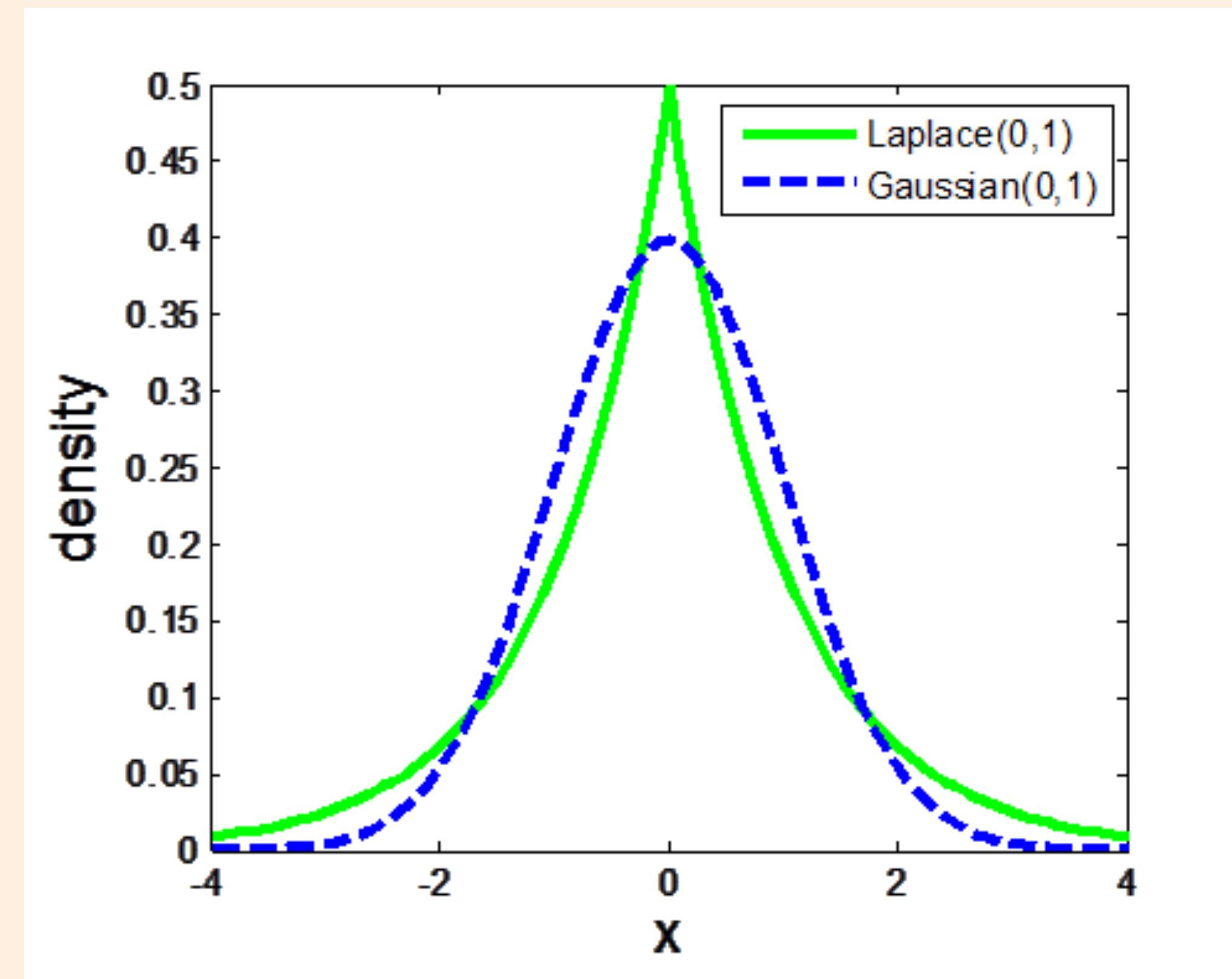
Different forms of regularization exist, which can also be expressed as Bayesian priors

Lasso Regression/ L_1 Regularization

- Instead of penalizing the objective according to the square of the parameter vector, use the absolute value instead.

- $$\hat{\beta} = \arg \min_{\beta} \left\{ \text{SSE}(\mathbf{y}; \mathbf{X}, \beta) + \gamma \sum_{d=0}^D |\beta_d| \right\}$$

- where γ is a constant penalty factor
- equivalent “Laplace”/“double exponential” prior distribution



There are some advantages and disadvantages to both Ridge and Lasso regression, but they're rather abstract and difficult to identify in a given task

Elastic Net regression combines both L_1 and L_2 regularization

- $\hat{\beta} = \arg \min_{\beta} \left\{ \text{SSE}(\mathbf{y}; \mathbf{X}, \beta) + \gamma_1 \sum_{d=0}^D |\beta_d| + \gamma_2 \sum_{d=0}^D \beta_d^2 \right\}$
- Adds an additional hyperparameter that can be tuned with cross-validation. The γ hyperparameters determine whether the L_1 or L_2 norm is more impactful on the overall objective.
- Can be reduced to Ridge or Lasso regression if either of the hyperparameters is equal to 0 - in practice, ENET tends to be more effective

Regularization and Bayesian Priors are trying to accomplish the same thing:

Prevent models from overfitting to the data by applying a penalty to the objective when the estimates to the parameters are far away from some expected value

- In Bayesian terms, we like to think of the prior as representing some information not contained in the data, or an initial hypothesis.
- In non-Bayesian terms, regularization represents a constraint on the size of the parameters.
- If we set the prior mean to 0, our normal-normal posterior reduces exactly to Ridge regression, where $1/\tau_0^2$ takes the place of the regularizing constant

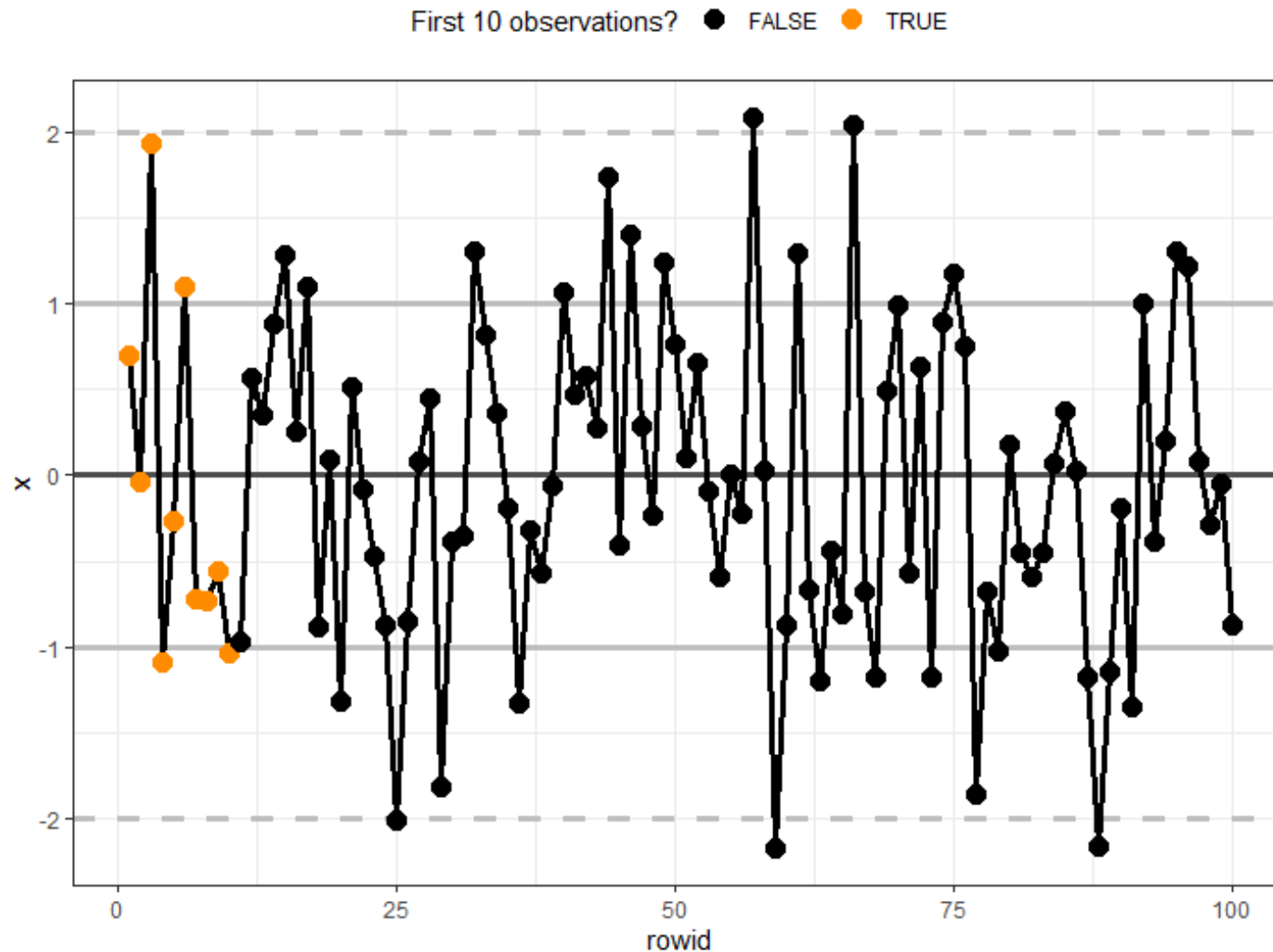
That's a lot of math...let's apply these ideas to a single input example

Synthetic data example

- Let's see if we can learn the parameters of a linear model.
- Data will be generated similar to what we did before.
- We will use different values of the intercept, slope, and noise compared to our previous visualizations.
- The input will be generated from a standard normal.

```
x <- rnorm(n = 100)
```

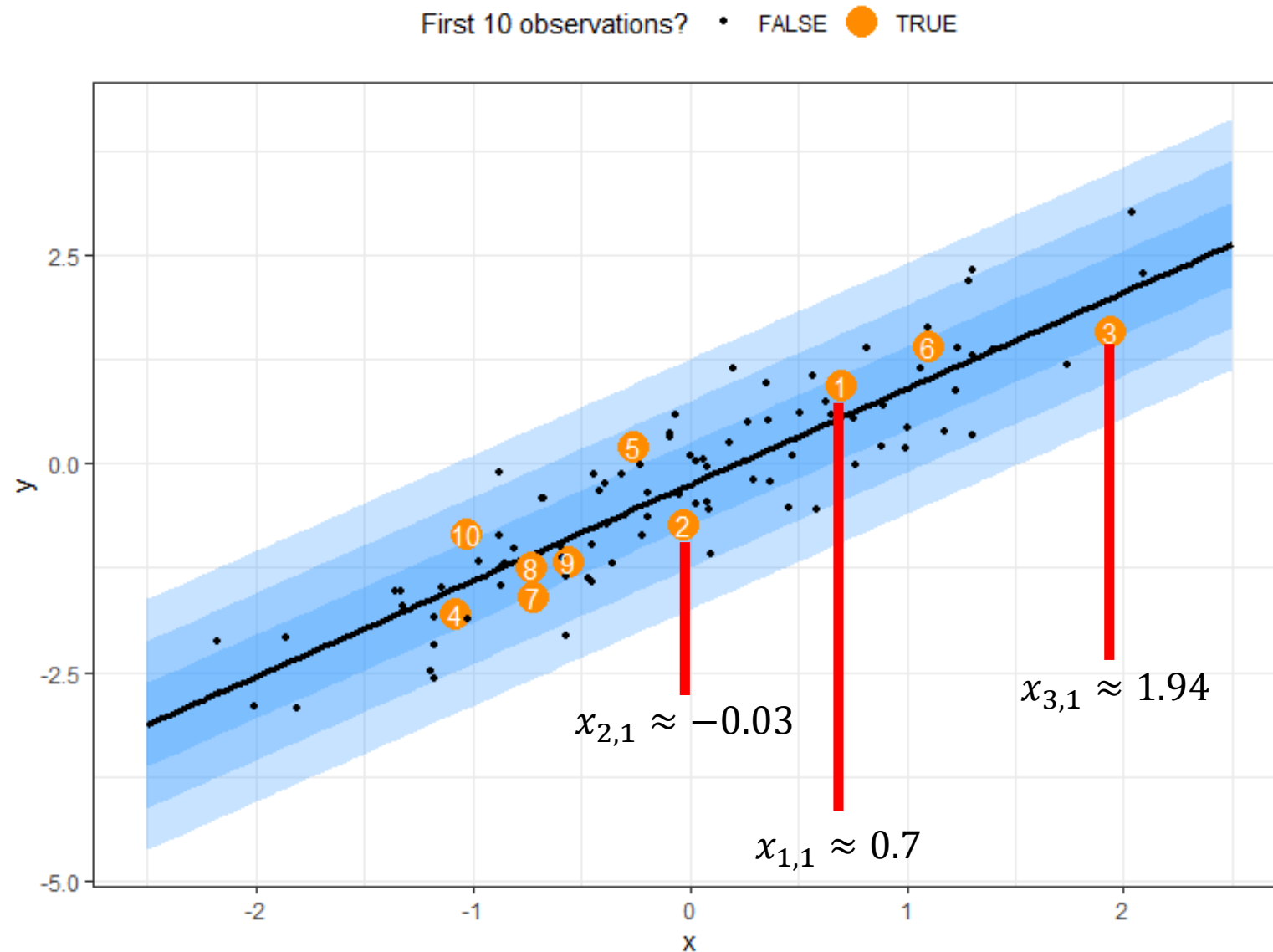
Generate 100 total random input values.



Specify true values for the parameters.

- Set $\beta_0 = -0.25, \beta_1 = 1.15$
- Calculate the true mean trend at each x_n .
- Set $\sigma = 0.5$
- Generate random observations based on the true mean trend and the specified noise.

The data points in the example



In R, we can quickly create design matrices using the `model.matrix()` function

- Assume the first 10 observations of `x` and `y` are stored in a data.frame named `train_df`.
- We can create the design matrix, `Xmat`, which includes the intercept column of 1s via:

```
Xmat <- model.matrix(y ~ x, data = train_df)
```

The formula interface creates the design matrix based on the formula NOT based on the columns in the data argument!

```
> train_df
# A tibble: 10 x 4
  obs_id      x      mu      y
  <int>   <dbl> <dbl> <dbl>
1     1  0.700  0.555  0.950
2     2 -0.0342 -0.289 -0.723
3     3  1.94   1.98   1.59
4     4 -1.09  -1.50  -1.79
5     5 -0.265  -0.555  0.196
6     6  1.10   1.01   1.42
7     7 -0.721  -1.08  -1.59
8     8 -0.728  -1.09  -1.24
9     9 -0.558  -0.892  -1.17
10    10 -1.03  -1.44  -0.848
> ## create the design matrix based on the first 10 points
> Xmat <- model.matrix(y ~ x, data = train_df)
> Xmat
  (Intercept)      x
1           1  0.69966425
2           1 -0.03418464
3           1  1.93918539
4           1 -1.08539595
5           1 -0.26491883
6           1  1.09797810
7           1 -0.72059350
8           1 -0.72799877
9           1 -0.55790203
10          1 -1.03392849
attr(,"assign")
[1] 0 1
```

train_df includes an index column
obs_id and the true mean trend mu.

The design matrix Xmat “understands”
to only use the x variable and that the
response was the y variable.

Let's look at the sum of squares matrix after the first observation

$$\mathbf{X}_{1,:}^T \mathbf{X}_{1,:}$$

$D+1 \times 1 \quad 1 \times D+1$

$$\begin{matrix} 1 & & 1 & 0.5 \\ 0.5 & & 0.5 & 0.25 \end{matrix}$$

- What are the dimensions after 1 observation?

$$(D + 1) \times (D + 1) = 2 \times 2$$

Let's look at the sum of squares matrix after the first observation

$$\mathbf{X}_{1,:}^T \mathbf{X}_{1,:} = \begin{bmatrix} x_{1,0}^2 & x_{1,0}x_{1,1} \\ x_{1,1}x_{1,0} & x_{1,1}^2 \end{bmatrix} = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 0.49 \end{bmatrix}$$

- The first observation of the zeroth variable: $x_{1,0} = 1$
- The first observation of the input: $x_{1,1} \approx 0.7$

What's the sum of squares matrix after 2 observations?

- What are the dimensions of the of the sum of squares after 2 observations?
- Still 2×2 !!!!

The sum of squares **ADDS** the contributions from the first and second observations:

$$\sum_{n=1}^{N=2} \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \mathbf{x}_{1,:}^T \mathbf{x}_{1,:} + \mathbf{x}_{2,:}^T \mathbf{x}_{2,:}$$

$$\sum_{n=1}^{N=2} \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \begin{bmatrix} x_{1,0}^2 & x_{1,0}x_{1,1} \\ x_{1,1}x_{1,0} & x_{1,1}^2 \end{bmatrix} + \begin{bmatrix} x_{2,0}^2 & x_{2,0}x_{2,1} \\ x_{2,1}x_{2,0} & x_{2,1}^2 \end{bmatrix}$$

The sum of squares **ADDS** the contributions from the first and second observations:

$$\sum_{n=1}^{N=2} \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \begin{bmatrix} x_{1,0}^2 & x_{1,0}x_{1,1} \\ x_{1,1}x_{1,0} & x_{1,1}^2 \end{bmatrix} + \begin{bmatrix} x_{2,0}^2 & x_{2,0}x_{2,1} \\ x_{2,1}x_{2,0} & x_{2,1}^2 \end{bmatrix}$$

- The second observation of the zeroth variable: $x_{2,0} = ?$

The sum of squares **ADDS** the contributions from the first and second observations:

$$\sum_{n=1}^{N=2} \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \begin{bmatrix} x_{1,0}^2 & x_{1,0}x_{1,1} \\ x_{1,1}x_{1,0} & x_{1,1}^2 \end{bmatrix} + \begin{bmatrix} x_{2,0}^2 & x_{2,0}x_{2,1} \\ x_{2,1}x_{2,0} & x_{2,1}^2 \end{bmatrix}$$

- The second observation of the zeroth variable: $x_{2,0} = 1$ ALWAYS!!

The sum of squares **ADDS** the contributions from the first and second observations:

$$\sum_{n=1}^{N=2} \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \begin{bmatrix} x_{1,0}^2 & x_{1,0}x_{1,1} \\ x_{1,1}x_{1,0} & x_{1,1}^2 \end{bmatrix} + \begin{bmatrix} x_{2,0}^2 & x_{2,0}x_{2,1} \\ x_{2,1}x_{2,0} & x_{2,1}^2 \end{bmatrix}$$

- The second observation of the zeroth variable: $x_{2,0} = 1$ ALWAYS!!
- The second observation of the input: $x_{2,1} \approx -0.03$

The sum of squares **ADDS** the contributions from the first and second observations:

$$\sum_{n=1}^{N=2} \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 0.49 \end{bmatrix} + \begin{bmatrix} 1 & -0.03 \\ -0.03 & 0.0009 \end{bmatrix} = \begin{bmatrix} 2 & 0.67 \\ 0.67 & 0.4909 \end{bmatrix}$$

- The second observation of the zeroth variable: $x_{2,0} = 1$ ALWAYS!!
- The second observation of the input: $x_{2,1} \approx -0.03$

The matrix operation takes care of the addition of ALL observations for us

$$\sum_{n=1}^N \{\mathbf{x}_{n,:}^T \mathbf{x}_{n,:}\} = \mathbf{X}^T \mathbf{X}$$

Let's calculate the posterior covariance matrix assuming a diffuse prior and 2 observations

- The known value of σ is stored in the variable `-sigma_true`

```
> Xmat[1:2, ,drop=FALSE]
  (Intercept)      x
1           1  0.69966425
2           1 -0.03418464
```

} Print out the first 2 rows from the DESIGN MATRIX

```
> t(Xmat[1:2, ,drop=FALSE]) %*% Xmat[1:2, ,drop=FALSE]
  (Intercept)      x
(Intercept)  2.0000000 0.6654796
x            0.6654796 0.4906986
```

} Print out the sum of squares matrix

```
> sigma_true^2 * solve( t(Xmat[1:2, , drop=FALSE]) %*% Xmat[1:2, , drop=FALSE] )
  (Intercept)      x
(Intercept)  0.2277936 -0.3089310
x            -0.3089310  0.9284462
```

} Print out the posterior covariance matrix

Let's calculate the posterior covariance matrix assuming a diffuse prior and 2 observations

- The known value of σ is stored in the variable `sigma_true`

```
> sigma_true^2 * solve( t(Xmat[1:2, , drop=FALSE]) %*% Xmat[1:2, , drop=FALSE] )
      (Intercept)          x
(Intercept)  0.2277936 -0.3089310
x            -0.3089310  0.9284462
```

- The posterior standard deviations are the square root of the main diagonal

```
> sqrt(diag(sigma_true^2 * solve( t(Xmat[1:2, , drop=FALSE]) %*% Xmat[1:2, , drop=FALSE] )))
      (Intercept)          x
      0.4772773    0.9635591
```

What about if we have an informative prior?

- Instead of $p(\boldsymbol{\beta}) \propto 1$...now let's use MVN prior with prior mean vector $\mathbf{b}_0$ and prior covariance matrix $\mathbf{B}_0$:

$$\boldsymbol{\beta} \mid \mathbf{b}_0, \mathbf{B}_0 \sim \mathcal{N}(\boldsymbol{\beta} \mid \mathbf{b}_0, \mathbf{B}_0)$$

Let's apply the informative prior calculations to the single input example

- Start with the first observation, print out the first row of the design matrix

```
> Xmat[1, ,drop=FALSE]  
(Intercept)      x  
1           1 0.6996642
```

Without the `drop=FALSE` flag, the result will be “dropped down” to a `numeric` from a `matrix` since the row-dimension is just 1. `drop=FALSE` forces the result to **STAY** as a `matrix`.

- Define the ~~prior covariance~~ matrix based on independent standard normals

```
> B0 <- diag(2)  
> B0  
      [,1] [,2]  
[1,]    1    0  
[2,]    0    1
```

Since the parameters are independent a-priori, the covariance matrix is a diagonal matrix. Can easily create a diagonal matrix with the `diag()` function, where we specify the number of rows and columns of the square matrix, and returns the identity matrix.

The posterior covariance matrix based on 1 observation and the informative prior

```
> solve(solve(B0) + (t(Xmat[1, ,drop=FALSE]) %*% Xmat[1, ,drop=FALSE])/sigma_true^2)
```

	(Intercept)	x
(Intercept)	0.4251321	-0.4022145
x	-0.4022145	0.7185849

Compare the informative prior result to NOT using a prior!

```
> solve(solve(B0) + (t(Xmat[1, ,drop=FALSE]) %*% Xmat[1, ,drop=FALSE])/sigma_true^2)
              (Intercept)              x
(Intercept)  0.4251321 -0.4022145
x            -0.4022145  0.7185849
```

- We cannot calculate the posterior covariance matrix between the 2 unknowns after **1 observations** when the prior is **truly** infinitely diffuse!

```
> sigma_true^2 * solve( t(Xmat[1, , drop=FALSE]) %*% Xmat[1, , drop=FALSE] )
Error in solve.default(t(Xmat[1, , drop = FALSE]) %*% Xmat[1, , drop = FALSE]) :
  Lapack routine dgesv: system is exactly singular: U[2,2] = 0
```

- We are trying to estimate 2 unknowns from 1 observation...cannot happen!

What if we had a diffuse prior with a large prior standard deviation: $\tau_{\beta} = 20$

- Even though it adds next to no information compared to the likelihood even the slightest information allows the posterior covariance matrix to be calculated when we have a single measurement!

```
> B0vague <- 20^2 * diag(2)
```

```
> B0vague
```

	[,1]	[,2]
[1,]	400	0
[2,]	0	400

Define the diffuse or vague prior covariance. A-priori the two parameters are still independent, but now the prior standard deviation is 20 or prior variance is 400!

What if we had a diffuse prior with a large prior standard deviation: $\tau_{\beta} = 20$

- Even though it adds next to no information compared to the likelihood even the slightest information allows the posterior covariance matrix to be calculated when we have a single measurement!

```
> B0vague <- 20^2 * diag(2)
> B0vague
      [,1] [,2]
[1,]  400   0
[2,]   0  400
```

Define the diffuse or vague prior covariance. A-priori the two parameters are still independent, but now the prior standard deviation is 20 or prior variance is 400!

- Calculate the posterior covariance matrix based on 1 observation and the diffuse prior!

```
> solve(solve(B0vague) + (t(Xmat[1, ,drop=FALSE]) %*% Xmat[1, ,drop=FALSE])/sigma_true^2)
      (Intercept)      x
(Intercept)  131.5716 -187.8098
x           -187.8098  268.5962
```

Look how large the posterior variances are compared to the case with the informative prior!

The diffuse prior results in higher posterior correlation between the parameters compared to the informative prior

- Use the `cov2cor()` function to convert from the covariance to a correlation matrix.
- The posterior correlation matrix based on the diffuse prior:

```
> cov2cor(solve(solve(B0vague) + (t(Xmat[1, ,drop=FALSE]) %*% Xmat[1, ,drop=FALSE])/sigma_true^2))
```

	(Intercept)	x
(Intercept)	1.0000000	-0.9990501
x	-0.9990501	1.0000000

Posterior correlation coefficient of -0.999!!!!!!

- The posterior correlation matrix based on the informative prior:

```
> cov2cor(solve(solve(B0) + (t(Xmat[1, ,drop=FALSE]) %*% Xmat[1, ,drop=FALSE])/sigma_true^2))
```

	(Intercept)	x
(Intercept)	1.0000000	-0.7277073
x	-0.7277073	1.0000000

Posterior correlation coefficient of -0.728

The posterior covariance matrix after 2 observations using the informative prior

```
> solve(solve(B0) + (t(Xmat[1:2, ,drop=FALSE]) %*% Xmat[1:2, ,drop=FALSE])/sigma_true^2)
      (Intercept)          x
(Intercept)  0.1513225 -0.1359555
x            -0.1359555  0.4596682
```

- Compare to the diffuse prior case with prior standard deviations equal to 20

```
> solve(solve(B0vague) + (t(Xmat[1:2, ,drop=FALSE]) %*% Xmat[1:2, ,drop=FALSE])/sigma_true^2)
      (Intercept)          x
(Intercept)  0.2274262 -0.3080403
x            -0.3080403  0.9260588
```

The posterior covariance matrix after 2 observations using the informative prior

```
> solve(solve(B0) + (t(Xmat[1:2, ,drop=FALSE]) %*% Xmat[1:2, ,drop=FALSE])/sigma_true^2)
      (Intercept)          x
(Intercept)  0.1513225 -0.1359555
x            -0.1359555  0.4596682
```

- Compare to the diffuse prior case with prior standard deviations equal to 20

```
> solve(solve(B0vague) + (t(Xmat[1:2, ,drop=FALSE]) %*% Xmat[1:2, ,drop=FALSE])/sigma_true^2)
      (Intercept)          x
(Intercept)  0.2274262 -0.3080403
x            -0.3080403  0.9260588
```

- As a reminder, the truly infinitely diffuse prior case posterior covariance matrix after 2 observations was:

```
> sigma_true^2 * solve( t(Xmat[1:2, , drop=FALSE]) %*% Xmat[1:2, , drop=FALSE] )
      (Intercept)          x
(Intercept)  0.2277936 -0.3089310
x            -0.3089310  0.9284462
```


In pictures....

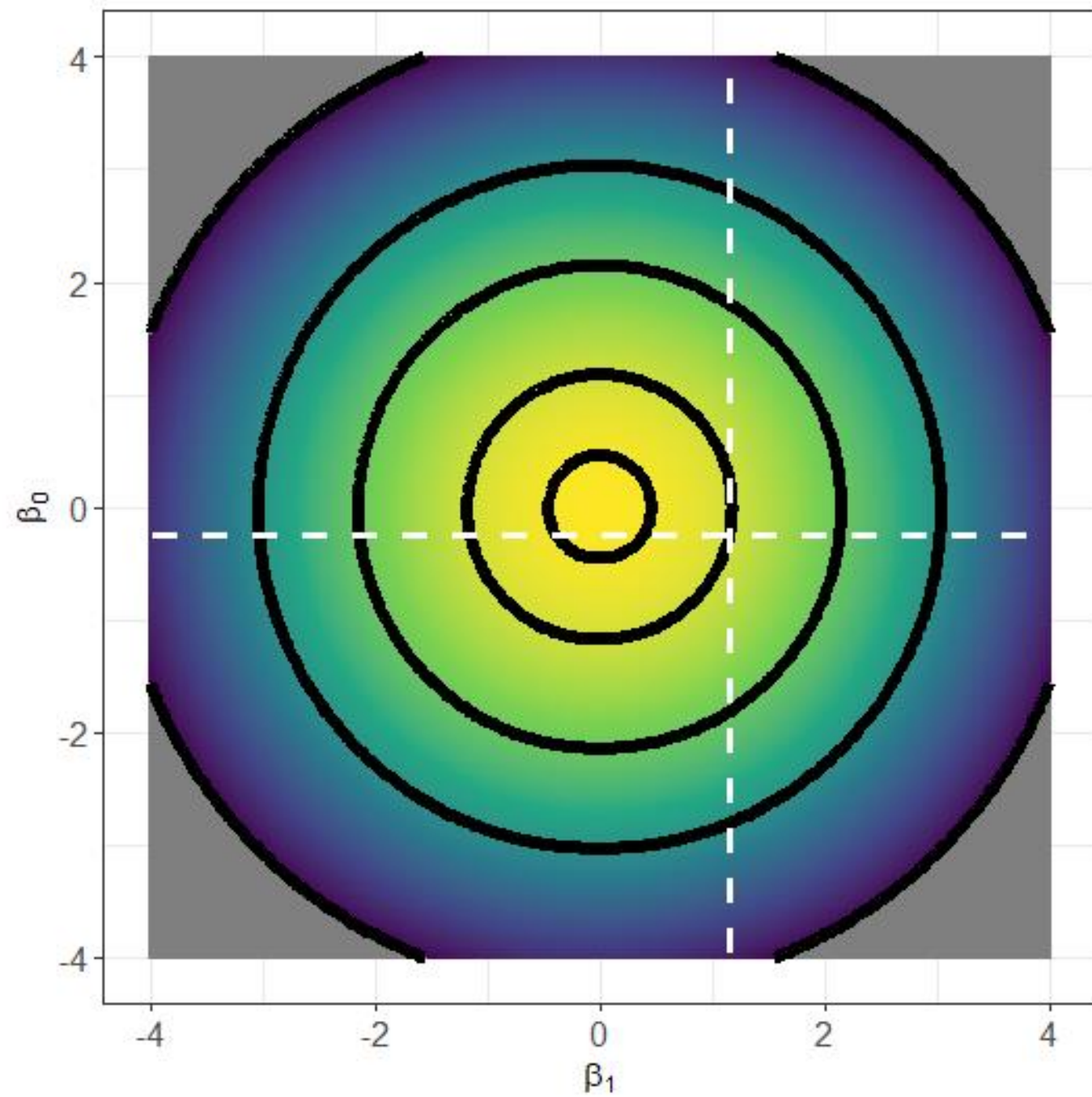
Informative Prior specification

- Let's assume that a-priori the parameters are independent:

$$p(\boldsymbol{\beta}) = p(\beta_0)p(\beta_1)$$

- Let's assign standard normal distributions to both parameters:

$$p(\boldsymbol{\beta}) = \text{normal}(\beta_0 \mid 0,1) \times \text{normal}(\beta_1 \mid 0,1)$$



Let's visualize the log-posterior surface based on the first 10 observations, the complete probability model is:

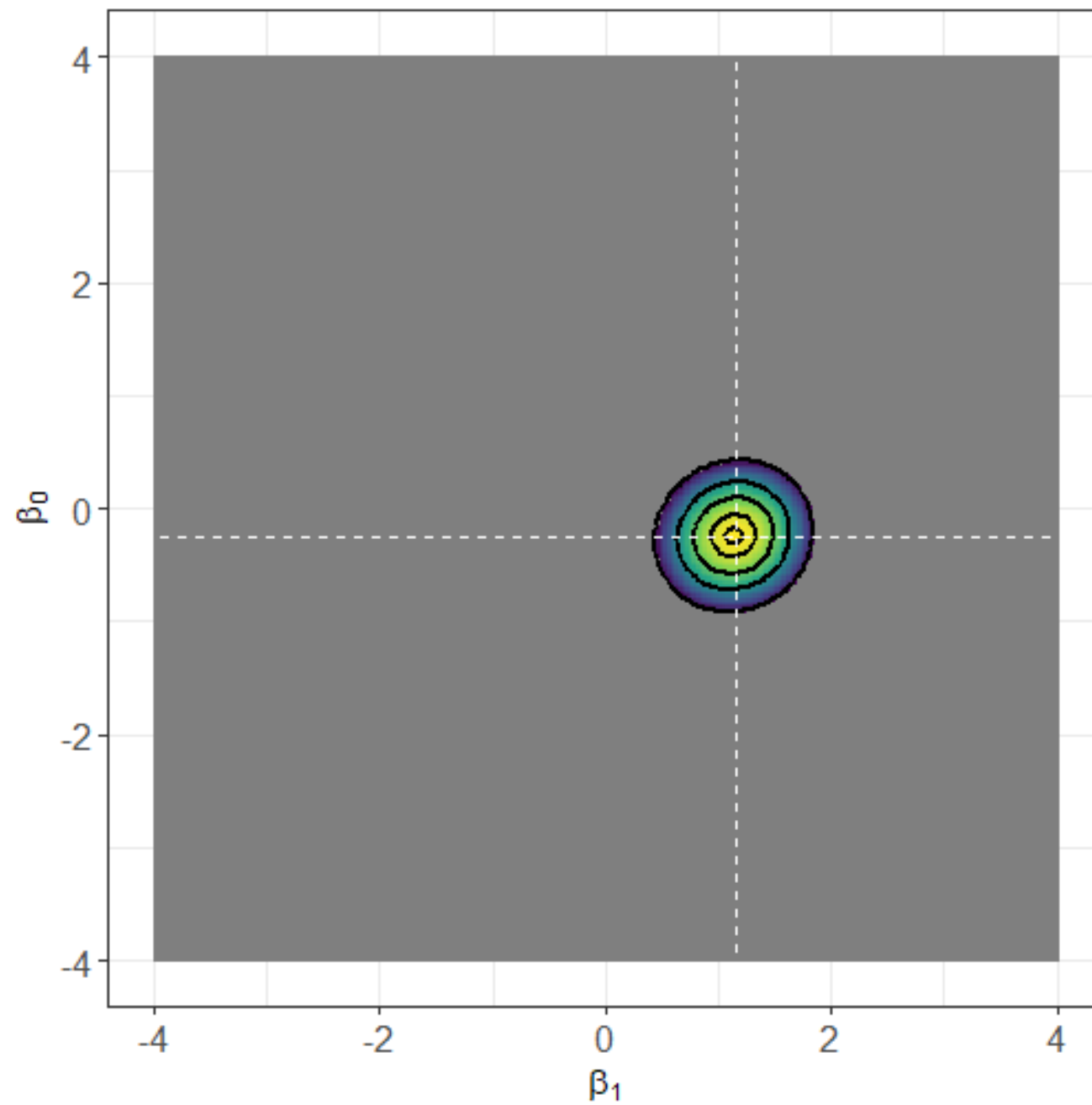
$$p(y_n \mid \mu_n, \sigma) \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \beta_0 + \beta_1 x_n$$

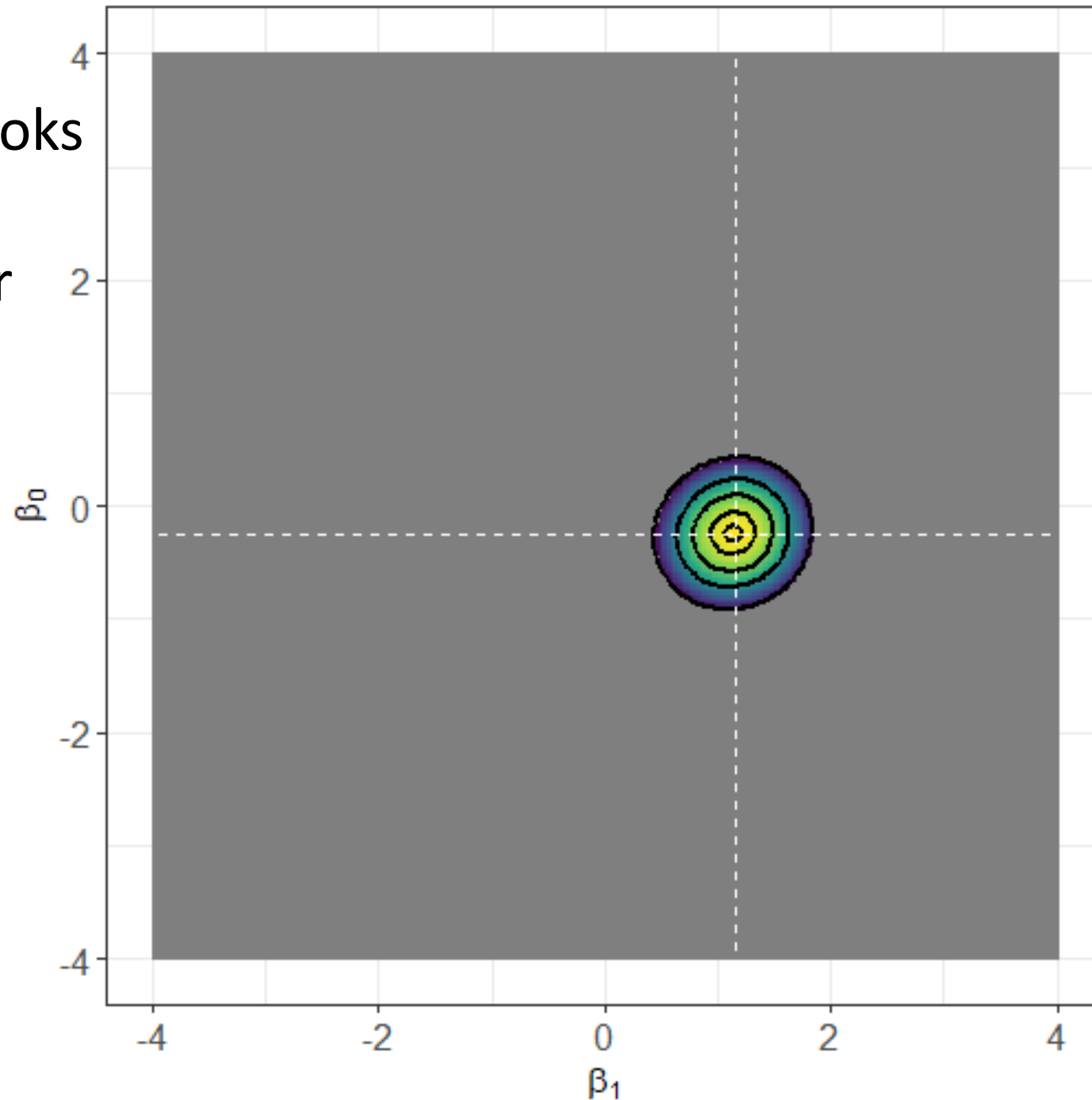
$$\beta_0 \sim \text{normal}(\beta_0 \mid 0, 1)$$

$$\beta_1 \sim \text{normal}(\beta_1 \mid 0, 1)$$

Log-posterior based on $N = 10$ observations

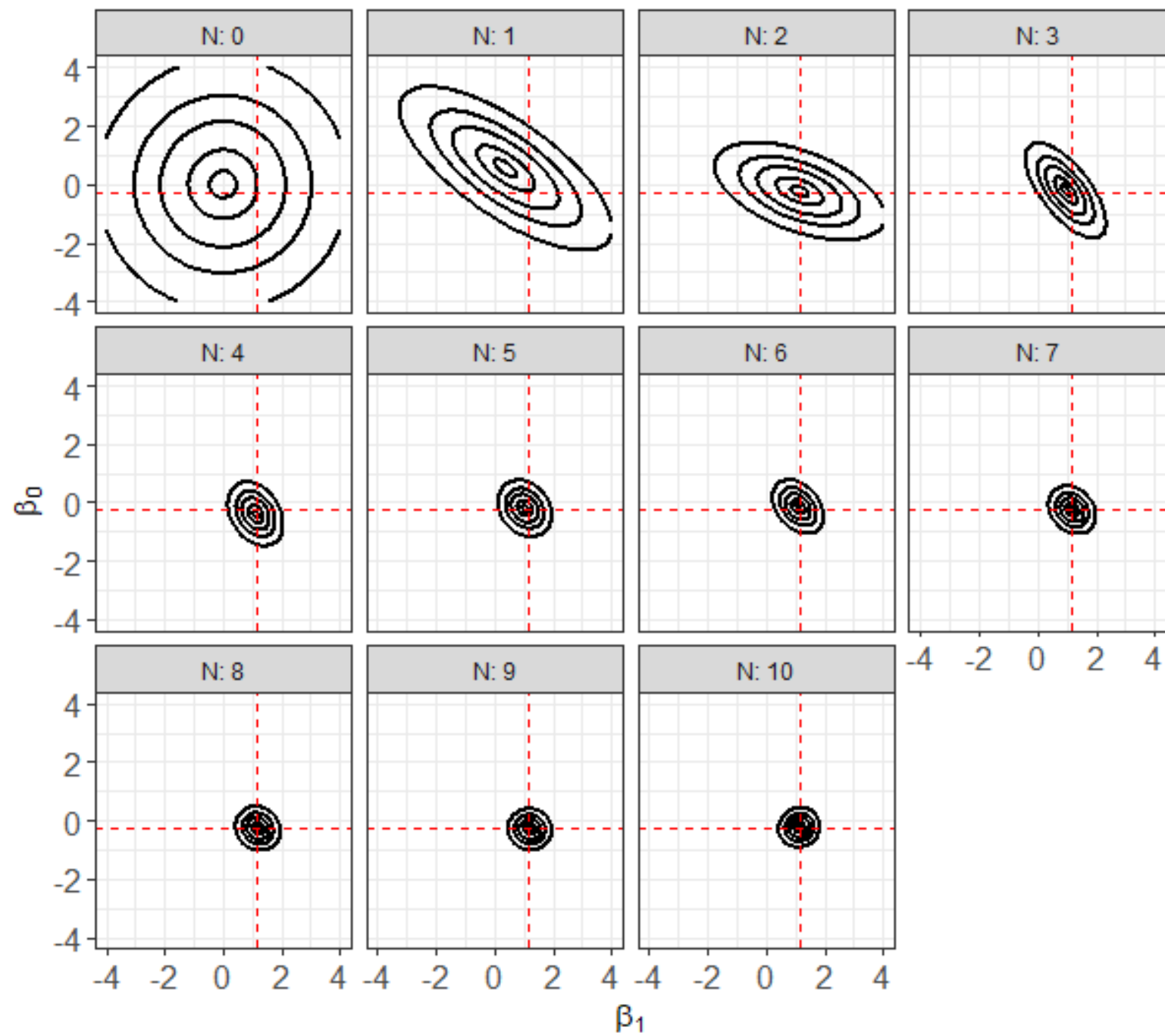


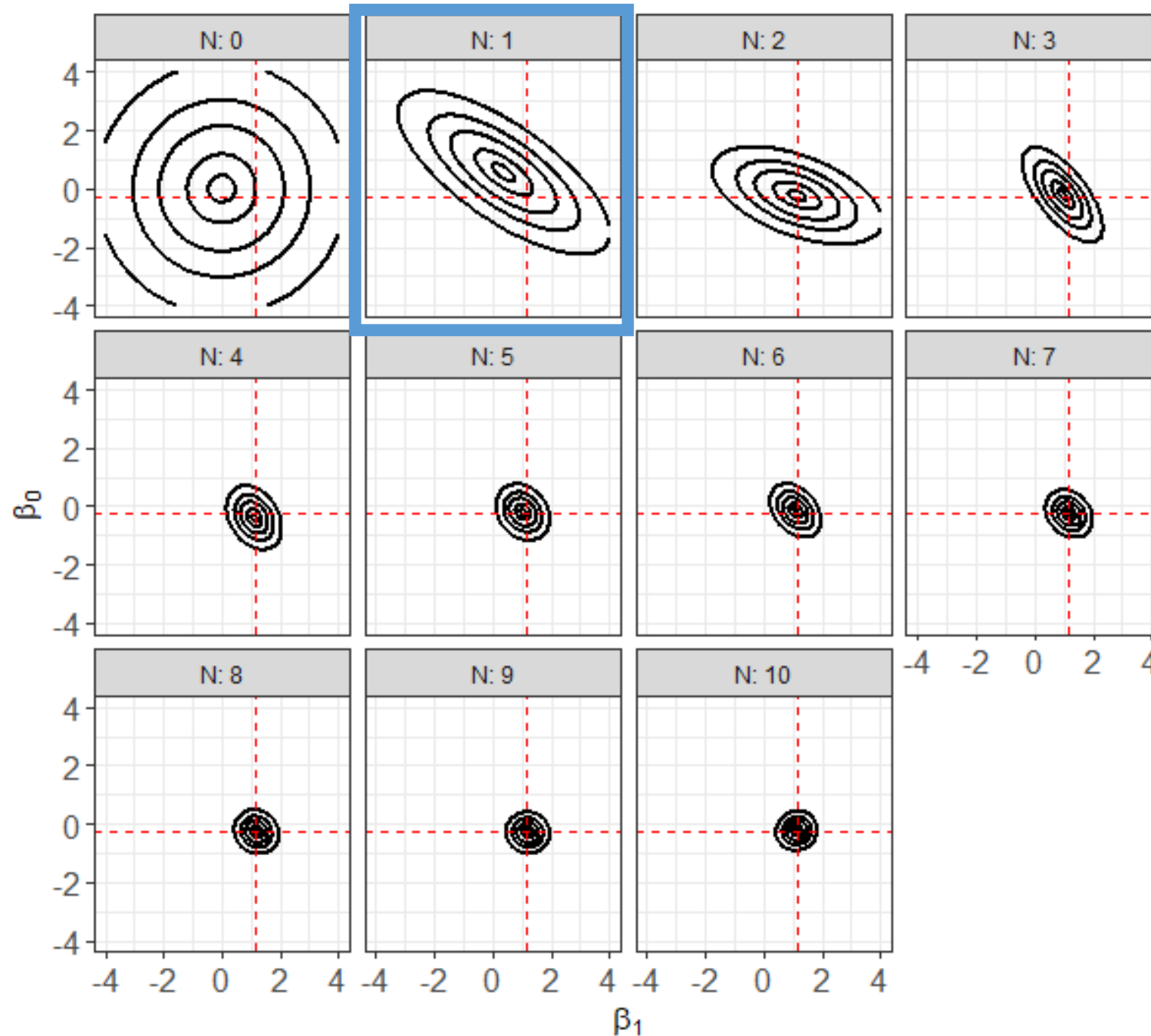
Log-posterior based on $N = 10$ observations



The posterior looks centered on the TRUE parameter values!!!

Posterior precision is VERY high!!

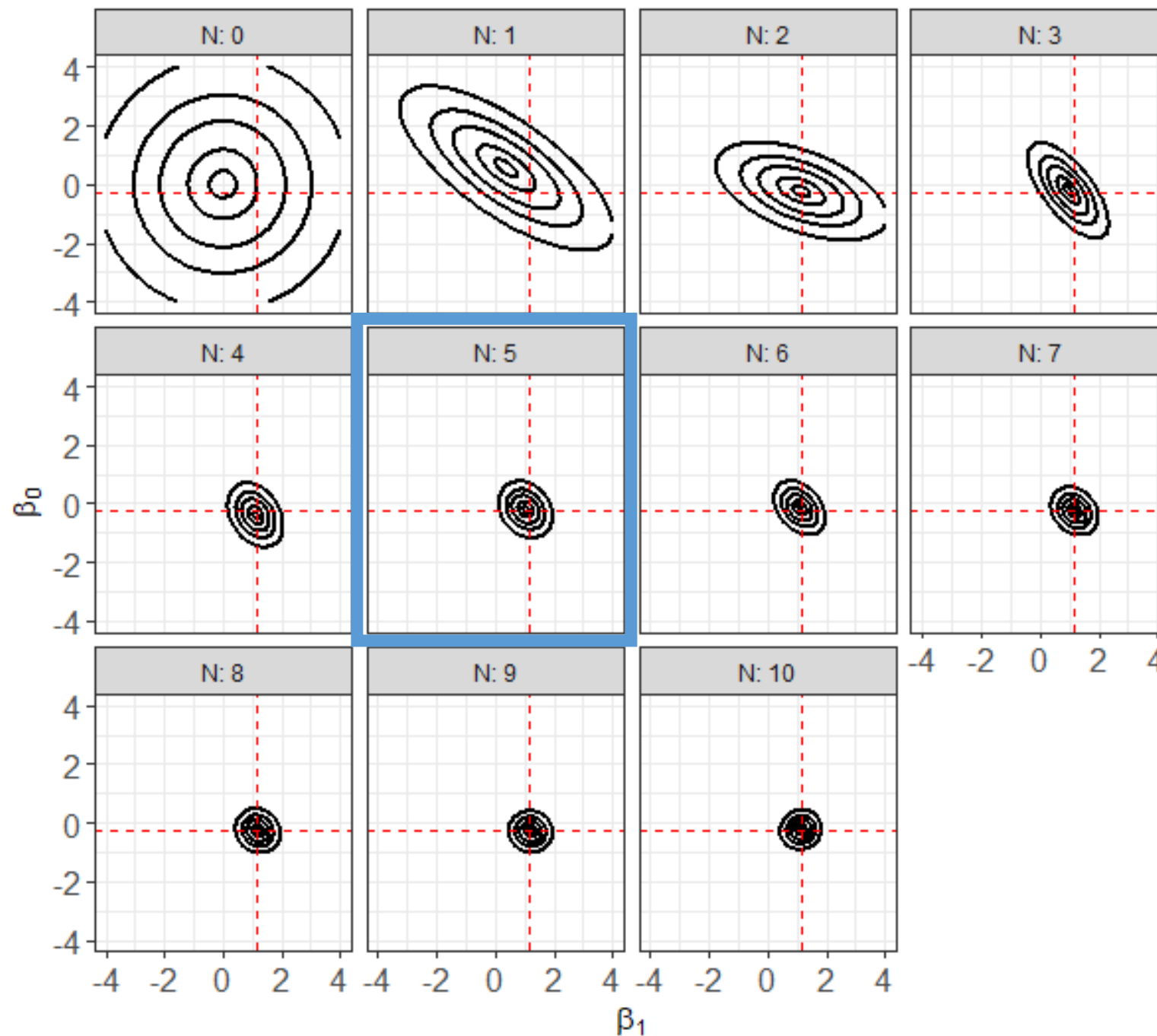




After just a single observation...we have updated our belief about 2 unknowns!!!!

The prior provides information just like when we were learning an unknown constant mean!

Are the parameters independent after 1 observation?



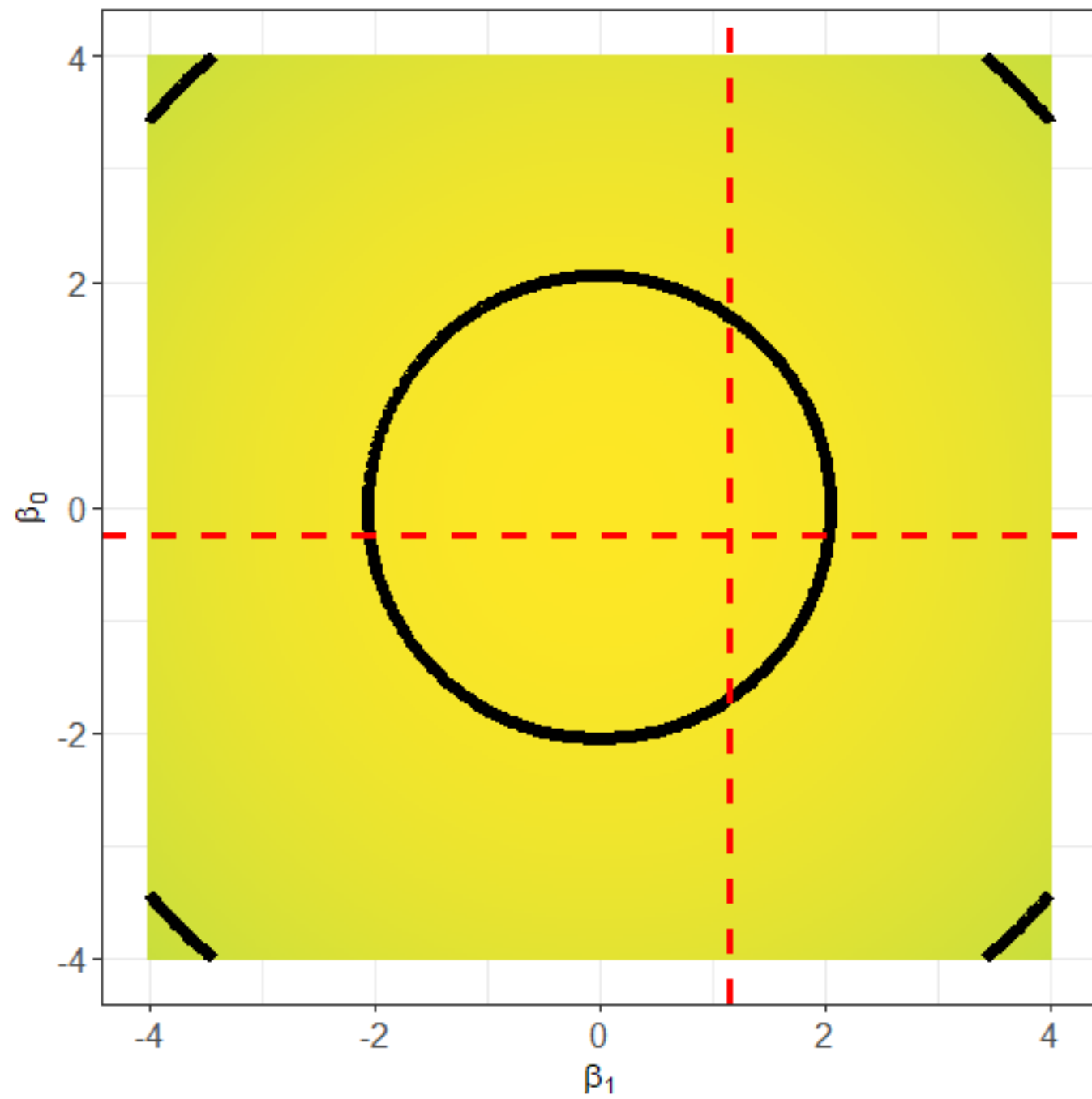
The posterior seems relatively unchanged after 5 observations...

Could the prior be influencing the posterior too much?

Try again, but this time with a diffuse prior

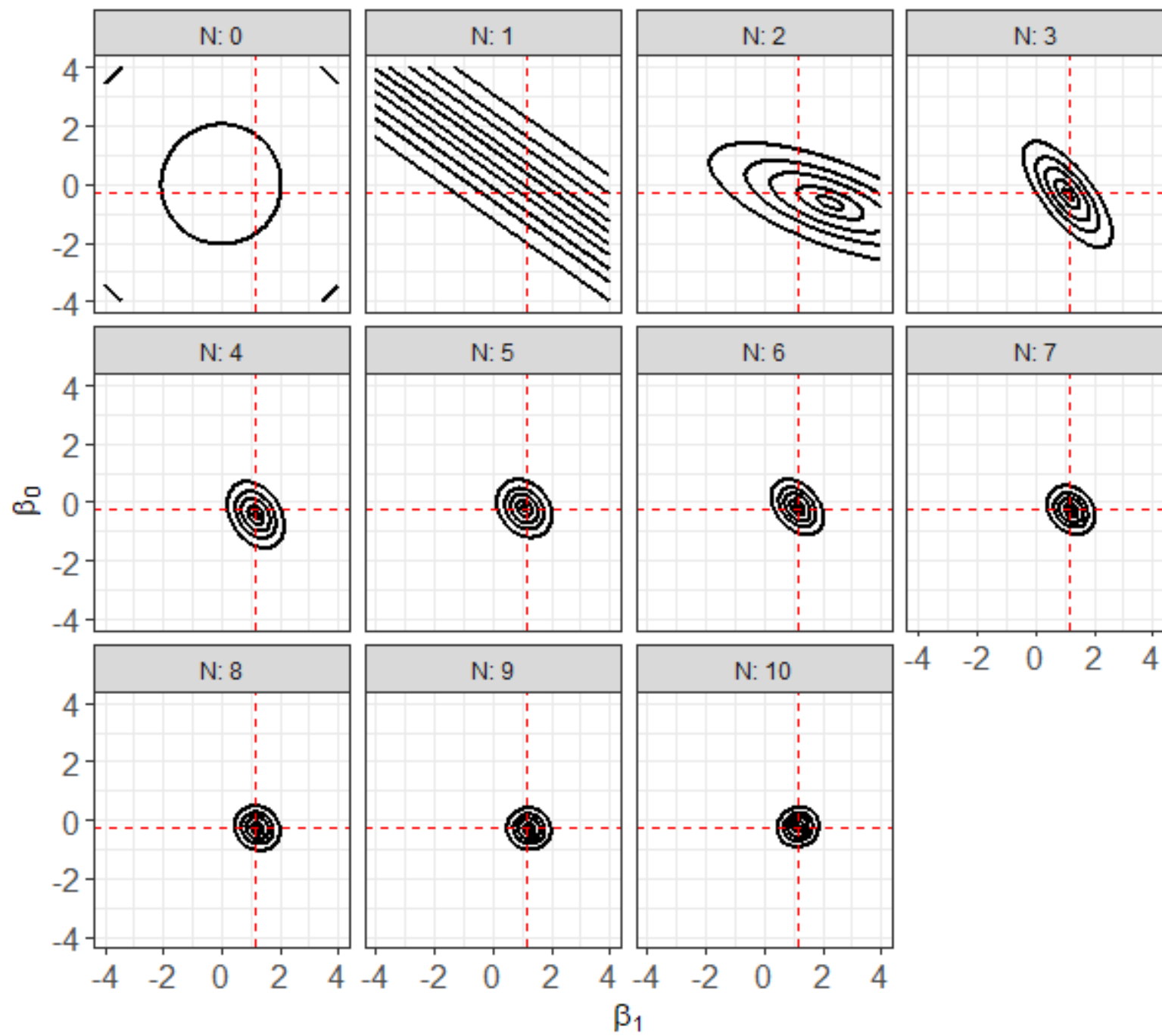
- Continue to assume the parameters are a-prior independent.
- But increase the prior standard deviations from 1 to 20.

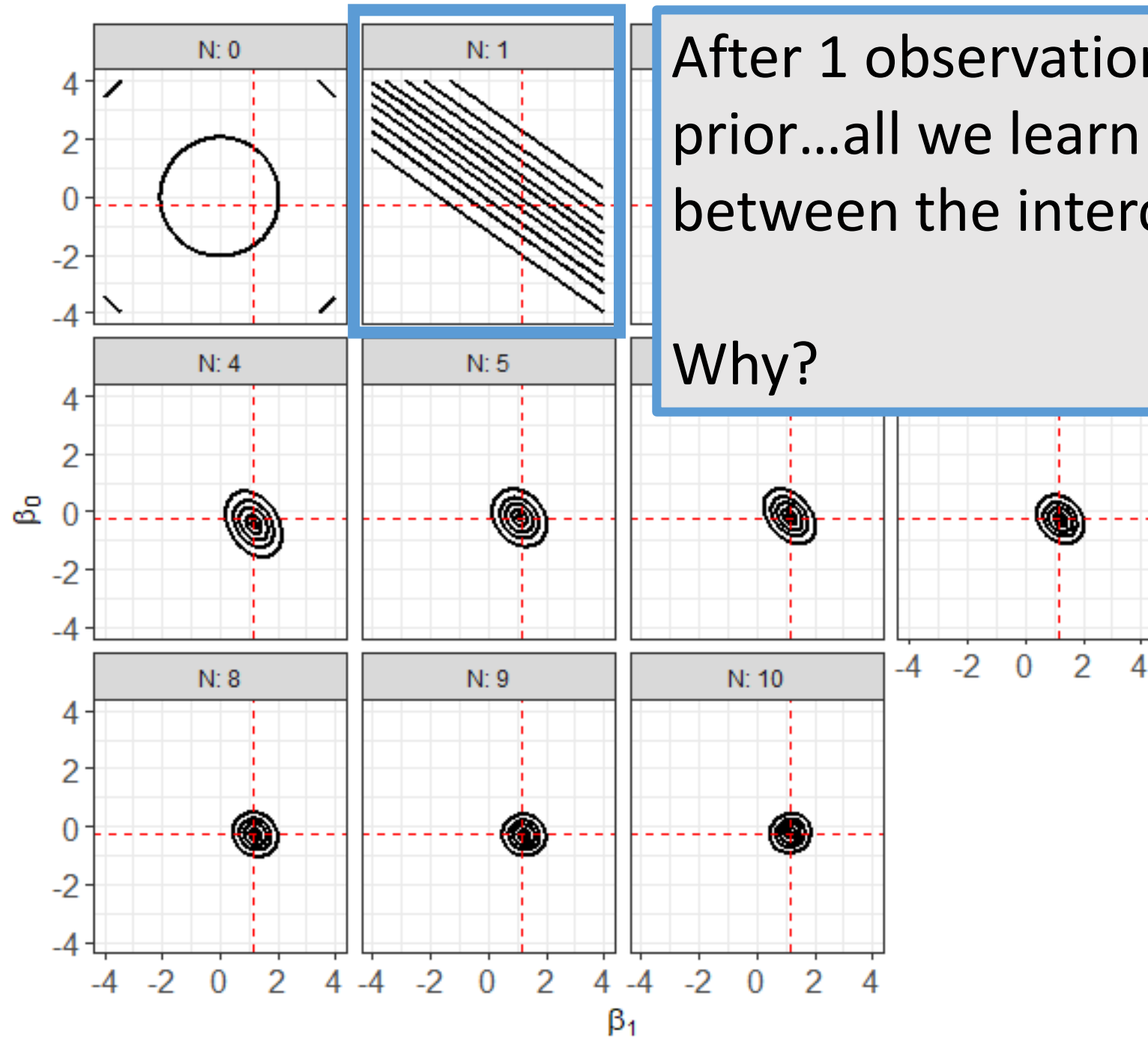
$$p(\beta) = \text{normal}(\beta_0 \mid 0, 20) \times \text{normal}(\beta_1 \mid 0, 20)$$



Repeat evaluating the log-posterior by sequentially adding the first 10 observations

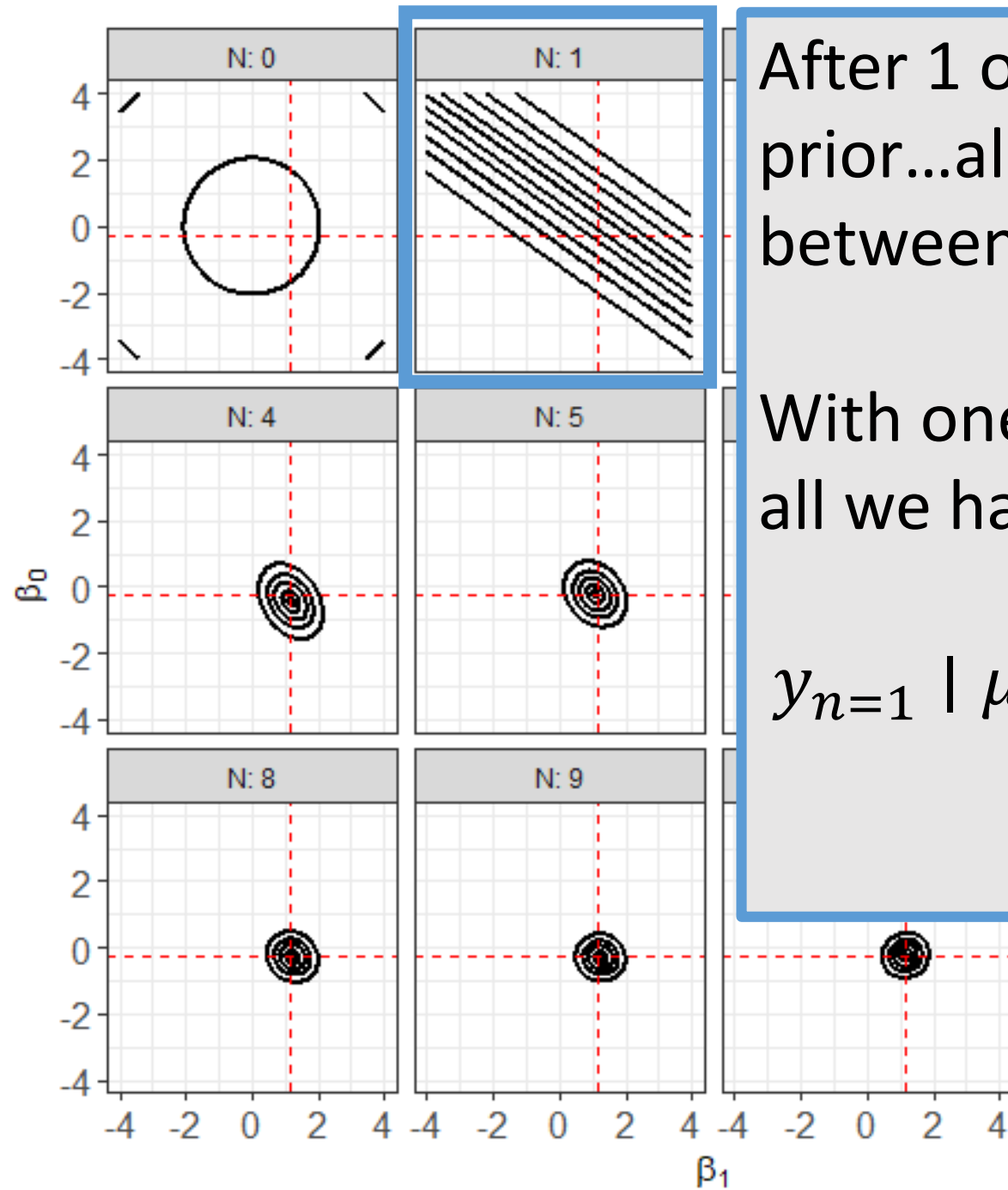
- Using such a diffuse prior forces the posterior to follow the likelihood.





After 1 observation with a diffuse prior...all we learn is a TRADE-OFF between the intercept and the slope!

Why?

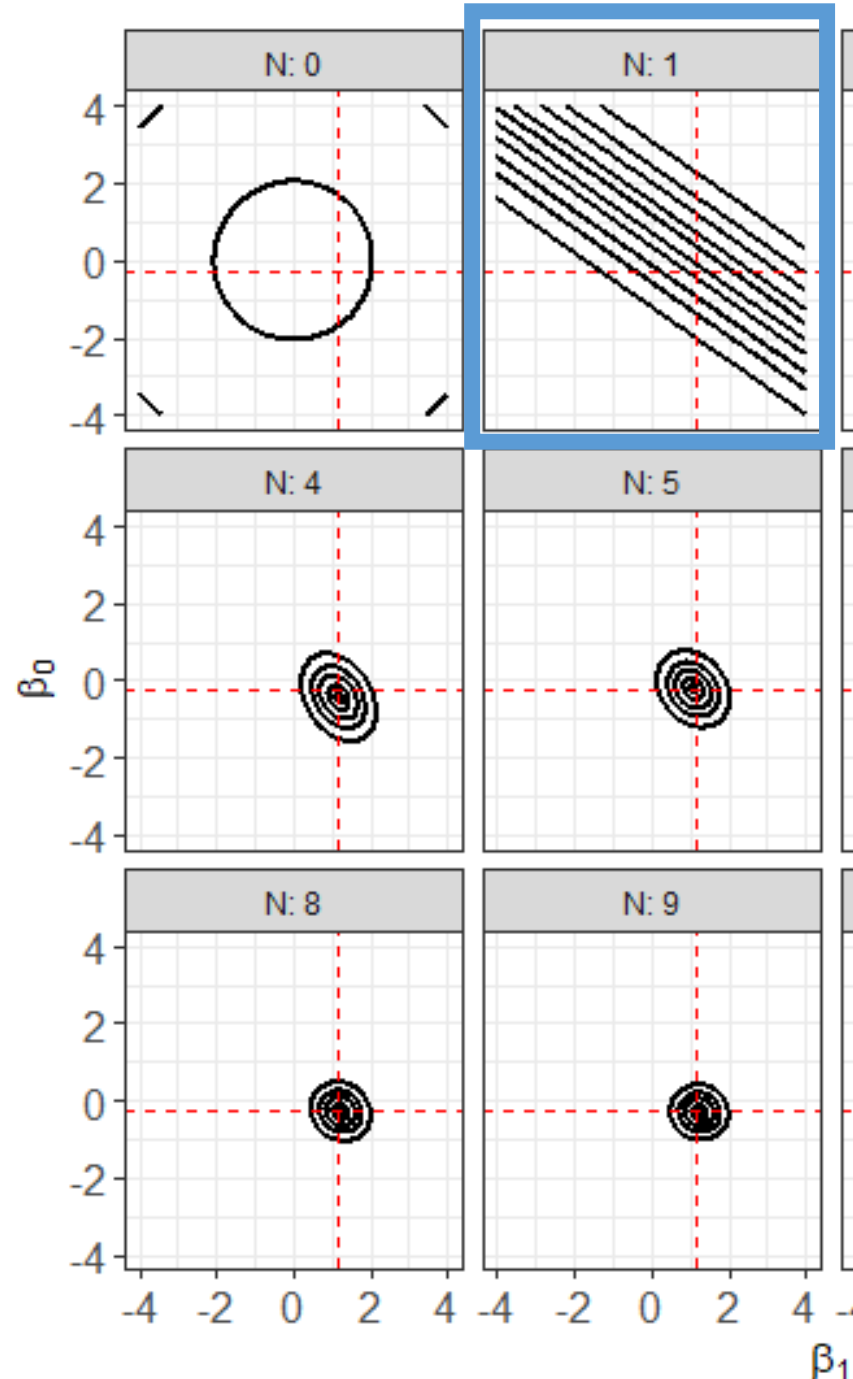


After 1 observation with a diffuse prior...all we learn is a TRADE-OFF between the intercept and the slope!

With one observation and a diffuse prior, all we have to consider is the likelihood:

$$y_{n=1} \mid \mu_{n=1}, \sigma \sim \text{normal}(y_{n=1} \mid \mu_{n=1}, \sigma)$$

$$\mu_{n=1} = \beta_0 + \beta_1 x_{n=1}$$



After 1 observation with a diffuse prior...all we learn is a TRADE-OFF between the intercept and the slope!

With one observation and a diffuse prior, all we have to consider is the likelihood:

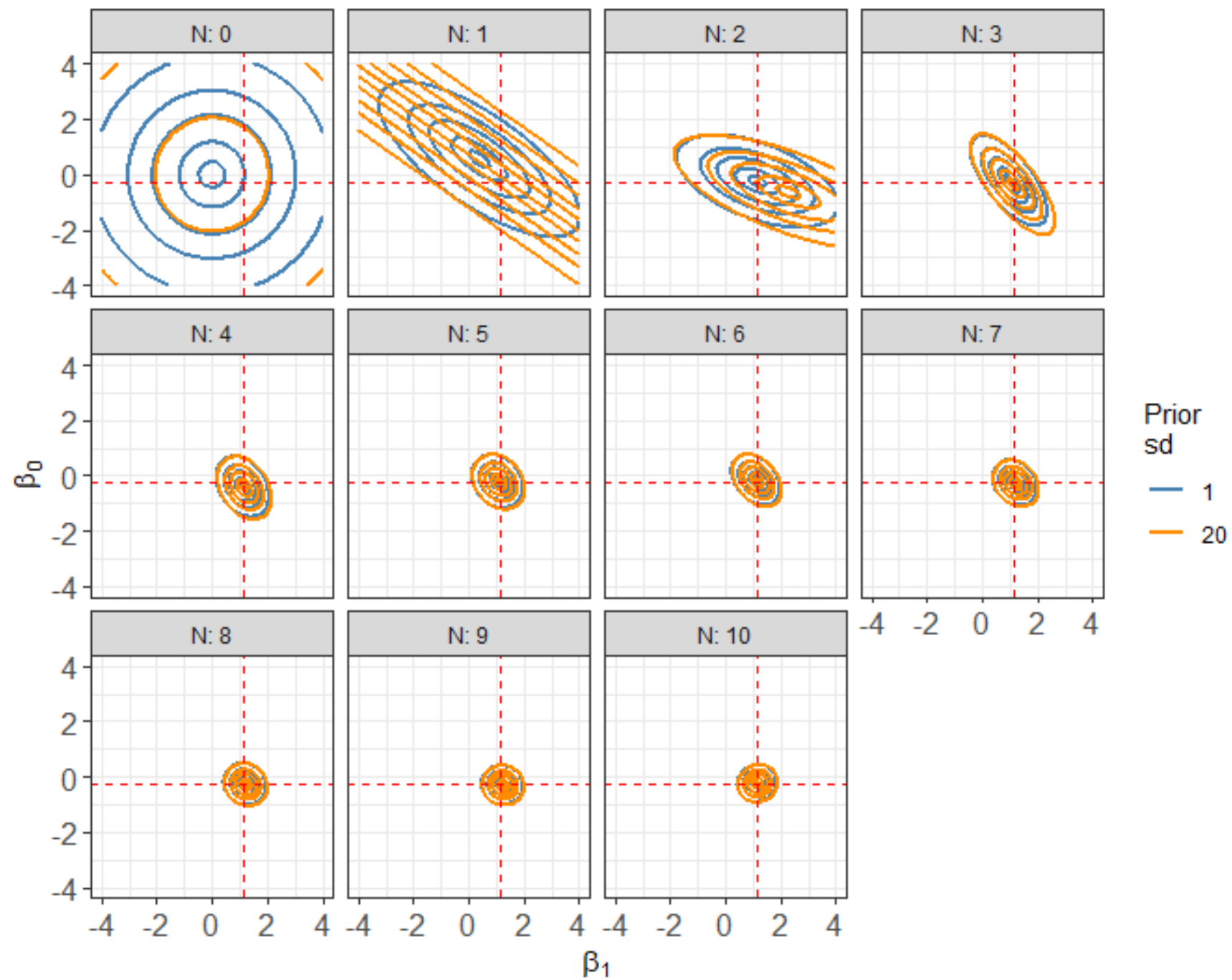
$$y_{n=1} \mid \mu_{n=1}, \sigma \sim \text{normal}(y_{n=1} \mid \mu_{n=1}, \sigma)$$

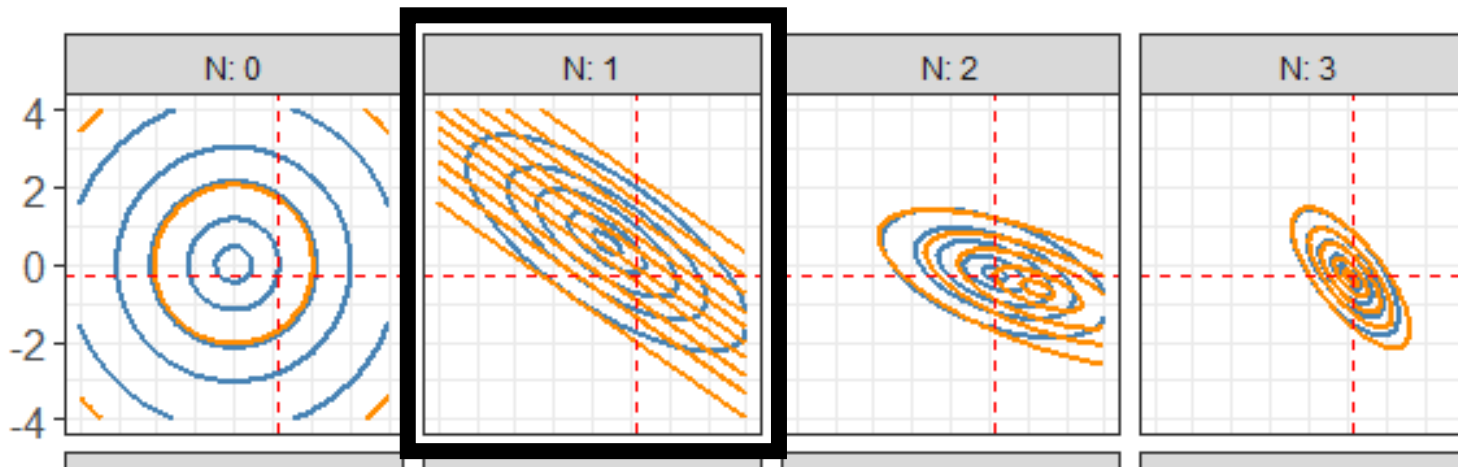
constant = $\uparrow$ $+$ $\downarrow$

$$\mu_{n=1} = \beta_0 + \beta_1 x_{n=1}$$

Increasing the slope, while decreasing the intercept will yield a constant value for $\mu_{n=1}$. The plotted contours represent that trade-off!

Directly compare the posterior based on the two priors as we add points.





After 1 observation, the informative prior allows the trade-off to still occur.

HOWEVER, the very **high slope** and very **high intercept** values have been **RULED OUT!**

The informative prior acts to **CONSTRAIN** the parameters away from **extreme** or large values!

Makes it easier to “see” the mode and reduces uncertainty!!!.

Everything we've covered so far...

Assumes σ is known...

- What happens when we don't know σ ?

Everything we've covered so far...
Assumes σ is known...

- What happens when we don't know σ ?
- We have to go back to the start!

Start from the log-likelihood again...or the log-posterior with an infinitely diffuse prior

$$\log[p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}, \sigma)] = \sum_{n=1}^N \left\{ -\frac{1}{2} \log[\sigma^2] - \frac{1}{2} \log[2\pi] - \frac{1}{2\sigma^2} (y_n - \mu_n)^2 \right\}$$

Start from the log-likelihood again...or the log-posterior with an infinitely diffuse prior

$$\log[p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}, \sigma)] = \sum_{n=1}^N \left\{ -\frac{1}{2} \log[\sigma^2] - \frac{1}{2} \log[2\pi] - \frac{1}{2\sigma^2} (y_n - \mu_n)^2 \right\}$$



Drop constants, simplify,
write in matrix notation

$$\log[p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}, \sigma)] \propto -\frac{N}{2} \log[\sigma^2] - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$$

Cannot drop
this term now!

Classical non-Bayesian approach: evaluate the log-likelihood at the coefficient MLEs

$$\log[p(\mathbf{y} \mid \mathbf{X}, \hat{\boldsymbol{\beta}}, \sigma)] \propto L \Big|_{\hat{\boldsymbol{\beta}}} = -\frac{N}{2} \log[\sigma^2] - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$$

- Take the derivative of L with respect to σ^2 :

$$\frac{\partial}{\partial(\sigma^2)} \left(L \Big|_{\hat{\boldsymbol{\beta}}} \right) = -\frac{N}{2} \frac{1}{\sigma^2} - \frac{1}{2} \left(\frac{1}{(\sigma^2)^2} \right) (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$$

Rearrange and simplify

$$\frac{\partial}{\partial(\sigma^2)} \left(L \mid_{\hat{\boldsymbol{\beta}}} \right) = \left(-\frac{1}{2\sigma^2} \right) \left(N - \frac{1}{\sigma^2} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) \right)$$

- The derivative will be zero at either infinite variance or if the second term on the RHS is equal to zero:

$$0 = N - \frac{1}{\hat{\sigma}^2} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$$

- Solve for $\hat{\sigma}^2$

The MLE $\hat{\sigma}^2$ is just the Mean Squared Error (MSE)!

$$\hat{\sigma}^2 = \frac{1}{N} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$$

- Statistics texts will show this in the more “correct” form, which divides by $N - (D + 1)$...but the concept is the same

In a Bayesian setting...it's just another unknown parameter!

- The posterior is now the joint posterior between the unknown model coefficients, $\boldsymbol{\beta}$, and the unknown noise, σ :

$$p(\boldsymbol{\beta}, \sigma \mid \mathbf{X}, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\beta}, \sigma) \times p(\boldsymbol{\beta}, \sigma)$$

- One option is to assume the noise is a-priori independent of the model coefficients:

$$p(\boldsymbol{\beta}, \sigma) = p(\boldsymbol{\beta})p(\sigma)$$

Complete model using an Exponential prior on σ with rate parameter $\lambda = 1$

$$p(y_n \mid \mu_n, \sigma) \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \beta_0 + \beta_1 x_n$$

$$\beta_0 \sim \text{normal}(\beta_0 \mid 0, 1)$$

$$\beta_1 \sim \text{normal}(\beta_1 \mid 0, 1)$$

$$\sigma \mid \lambda = 1 \sim \text{Exp}(\sigma \mid \lambda = 1)$$

Complete model using an Exponential prior on σ with rate parameter $\lambda = 1$

$$p(y_n \mid \mu_n, \sigma) \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

$$\mu_n = \beta_0 + \beta_1 x_n$$

$$\beta_0 \sim \text{normal}(\beta_0 \mid 0, 1)$$

$$\beta_1 \sim \text{normal}(\beta_1 \mid 0, 1)$$

$$\sigma \mid \lambda = 1 \sim \text{Exp}(\sigma \mid \lambda = 1)$$

What do you think will need to be applied in order to perform the Laplace Approximation to this case?